

Multimodal Autonomous Agent Application

Multimodal Autonomous Agent Application

1. Concept Introduction
 - 1.1 What is an "Autonomous Agent"?
 - 1.2 Brief Description of the Principle
2. Code Analysis
 - Key Code
 1. Agent Core Workflow (`targetmodel/utis/ai_agent.py`)
 2. Task Planning and LLM Interaction (`targetmodel/utis/ai_agent.py`)
 3. Parameter Processing and Data Flow Implementation (`targetmodel/utis/ai_agent.py`)
 - Code Analysis
3. Practical Operation
 - 3.1 Configure Offline Large Model
 - 3.1.1 Configure LLM Platform (`yahboom.yaml`)
 - 3.1.2 Configure Model Interface (`large_model_interface.yaml`)
 - 3.2 Start and Test the Function (Text Input Mode)
4. Common Problems and Solutions
 - 4.1 Abnormal Agent Behavior
 - Problem 1: The agent is stuck in an infinite loop or repeatedly executes the same tool.

Note: The RDK X5 4GB version cannot run this offline due to performance limitations. Please refer to the tutorial for online large models.

1. Concept Introduction

1.1 What is an "Autonomous Agent"?

In the `targetmodel` project, a **multimodal autonomous agent** is the highest form of intelligence. It no longer simply responds to a single user command, but can **autonomously think, plan, and continuously call multiple tools to complete a complex goal**.

The core of this function is the `agent_call` tool or the `ToolChainManager` behind it. When the user makes a complex request that cannot be completed with a single tool call, the autonomous agent is activated.

1.2 Brief Description of the Principle

The implementation of the autonomous agent in `targetmodel` follows the mainstream **ReAct (Reason + Act)** paradigm in the industry. Its core idea is to imitate the human problem-solving process, cycling between "thinking" and "acting."

1. **Reason:** When the agent receives a complex goal, it first calls a powerful language model (LLM) to "think." It asks itself: "To complete this goal, what should my first step be? Which tool should I use?" The LLM's output is not the final answer, but an action plan.
2. **Act:** The agent executes the corresponding action based on the LLM's thinking result—calling `ToolsManager` to run the specified tool (such as `visual_positioning`).

3. **Observe:** The agent obtains the result of the previous action ("observation"), for example, `{"result": "Found the cup, position is at [120, 300, 180, 360]"}`.
4. **Reason Again:** The agent submits this observation, along with the original goal, to the LLM again for a second round of "thinking." It asks itself: "I have found the position of the cup, what should I do next to know its color?" The LLM may generate a new action plan, such as `{"thought": "I need to analyze the image of the cup's area to determine the color", "action": "seewhat", "args": {"crop_area": [120, 300, 180, 360]}}`.

This **Reason -> Act -> Observe** cycle continues until the original goal is achieved, at which point the agent generates and outputs the final answer.

2. Code Analysis

Key Code

1. Agent Core Workflow (`largemodel/utils/ai_agent.py`)

The `_execute_agent_workflow` function is the main execution loop of the Agent, which defines the core process of "Plan -> Execute."

```
# From largemodel/utils/ai_agent.py

class AIAgent:
    # ...

    def _execute_agent_workflow(self, task_description: str) -> Dict[str, Any]:
        """
        Executes the agent workflow: Plan -> Execute.
        """
        try:
            # Step 1: Task Planning
            self.node.get_logger().info("AI Agent starting task planning phase")
            plan_result = self._plan_task(task_description)

            # ... (Return early if planning fails)

            self.task_steps = plan_result["steps"]

            # Step 2: Execute all steps in order
            execution_results = []
            tool_outputs = []

            for i, step in enumerate(self.task_steps):
                # 2.1. Process data references in parameters before execution
                processed_parameters =
self._process_step_parameters(step.get("parameters", {}), tool_outputs)
                step["parameters"] = processed_parameters

                # 2.2. Execute a single step
                step_result = self._execute_step(step, tool_outputs)
                execution_results.append(step_result)

                # 2.3. If the step is successful, save its output for subsequent
                steps to reference
```

```

        if step_result.get("success") and step_result.get("tool_output"):
            tool_outputs.append(step_result["tool_output"])
        else:
            # If any step fails, terminate the entire task
            return { "success": False, "message": f"Task terminated
because step '{step['description']}' failed." }

        # ... Summarize and return the final result
        summary = self._summarize_execution(task_description,
execution_results)
        return { "success": True, "message": summary, "results":
execution_results }

    # ... (Exception handling)

```

2. Task Planning and LLM Interaction (`largemodel/utils/ai_agent.py`)

The core of the `_plan_task` function is to construct a sophisticated Prompt that utilizes the reasoning ability of the large model itself to generate a structured execution plan.

```

# From largemodel/utils/ai_agent.py

class AIAgent:
    # ...
    def _plan_task(self, task_description: str) -> Dict[str, Any]:
        """
        Uses the large model for task planning and decomposition.
        """
        # Dynamically generate a list of available tools and their descriptions
        tool_descriptions = []
        for name, adapter in self.tools_manager.tool_chain_manager.tools.items():
            # ... (Get tool description from adapter.input_schema)
            tool_descriptions.append(f"- {name}({params}): {description}")
        available_tools_str = "\\n".join(tool_descriptions)

        # Construct a highly structured planning Prompt
        planning_prompt = f"""
As a professional task planning Agent, please decompose the user task into a
series of specific, executable JSON steps.

*** Available Tools:***
{available_tools_str}

*** Core Rules:***
1.  **Data Transfer**: When a subsequent step needs to use the output of a
previous step, you must use the `{{{steps.N.outputs.KEY}}}` format for
reference.
    - `N` is the step ID (starting from 1).
    - `KEY` is the specific field name in the output data of the previous step.
2.  **JSON Format**: You must strictly return a JSON object.

*** User Task:***
{task_description}
"""

```

```

# Call the large model for planning
messages_to_use = [{"role": "user", "content": planning_prompt}]
# Note that the general text inference interface is called here
result = self.node.model_client.infer_with_text("",
message=messages_to_use)

# ... (Parse the JSON response and return the list of steps)

```

3. Parameter Processing and Data Flow Implementation (`largemodel/utils/ai_agent.py`)

The `_process_step_parameters` function is responsible for parsing placeholders and implementing data flow between steps.

```

# From largemodel/utils/ai_agent.py

class AIAgent:
    # ...
    def _process_step_parameters(self, parameters: Dict[str, Any],
previous_outputs: List[Any]) -> Dict[str, Any]:
    """
    Parses parameter dictionary, finds and replaces all {{...}} references.
    """
    processed_params = parameters.copy()
    # Regular expression to match placeholders in the format
    {{steps.N.outputs.KEY}}
    pattern = re.compile(r"\{\{\steps\\.(\\d+)\\.outputs\\.(.+?)\\}\\}")

    for key, value in processed_params.items():
        if isinstance(value, str) and pattern.search(value):
            # Use re.sub and a replacement function to handle all found
            placeholders
            # The replacement function will find and return the value from
            the previous_outputs list
            processed_params[key] = pattern.sub(replacer_function, value)

    return processed_params

```

Code Analysis

The AI Agent is the "central brain" of the system. It transforms high-level, sometimes vague, user tasks into a series of precise, orderly tool calls. Its implementation does not depend on any specific model platform, but is built on a general, extensible architecture.

1. **Dynamic Task Planning:** The core capability of the Agent lies in the `_plan_task` function. It does not rely on hard-coded logic, but dynamically generates a task plan by interacting with the large model.
 - **Self-awareness and Prompt Construction:** At the beginning of planning, the Agent first checks all currently available tools and their descriptions. Then, it packages this tool information, the user task, and strict rules (such as data transfer format) into a highly structured `planning_prompt`.

- **Model as Planner:** This Prompt is sent to a general text-based large model. The model performs reasoning based on the provided context and returns an action plan in JSON format with multiple steps. This design is highly extensible: when new tools are added or modified in the system, the Agent's planning ability is automatically updated without modifying the Agent's own code.
2. **Tool Chain and Data Flow:** Real-world tasks often require the collaboration of multiple tools, such as "take a picture and describe it," which requires the output of the "take a picture" tool (image path) to be used as the input of the "describe" tool. The AI Agent elegantly implements this through the `_process_step_parameters` function.
- **Data Reference Placeholders:** During the planning phase, the large model embeds special placeholders, such as `{{steps.1.outputs.data}}`, in the parameter values where data needs to be passed.
 - **Real-time Parameter Replacement:** In the main loop of `_execute_agent_workflow`, `_process_step_parameters` is called before executing each step. It uses regular expressions to scan all parameters of the current step. Once a placeholder is found, it finds the corresponding data from the output list of the previous step and performs real-time replacement. This mechanism is the key to achieving the automation of complex tasks.
3. **Supervised Execution and Fault Tolerance:** `_execute_agent_workflow` constitutes the main execution loop of the Agent. It strictly follows the planned sequence of steps, executes each action in turn, and ensures that data is correctly passed between steps.
- **Atomic Steps:** Each step is treated as an independent "atomic operation." If any step fails, the entire task chain is immediately terminated and an error is reported. This ensures the stability and predictability of the system, avoiding continued execution in an error state.

In summary, the general implementation of the AI Agent demonstrates an advanced software architecture: it does not directly solve the problem, but builds a framework that allows an external, general-purpose reasoning engine (the large model) to solve the problem. Through the two core mechanisms of "dynamic planning" and "data flow management," the Agent can orchestrate a series of independent tools into complex workflows capable of completing high-level tasks.

3. Practical Operation

3.1 Configure Offline Large Model

3.1.1 Configure LLM Platform (`yahboom.yaml`)

This file determines which large model platform the `model_service` node loads as its primary language model.

1. **Open the file in the terminal:**

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

2. **Modify/Confirm `llm_platform`:**

```

model_service:                                # Model server node parameters
  ros__parameters:
    language: 'en'                            # Large model interface language
    useolinetts: True                         # This item is invalid in text mode
    and can be ignored

    # Large model configuration
    llm_platform: 'ollama'                    # Key: Make sure this is 'ollama'
    regional_setting : "China"
    camera_type: 'csi'                        # Camera type: 'csi', 'usb'

```

3.1.2 Configure Model Interface (large_model_interface.yaml)

This file defines which vision model to use when the platform is selected as `ollama`.

1. Open the file in the terminal

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

2. Find the ollama-related configuration

```

#.....
## Offline Large Language Models
# ollama configuration
ollama_host: "http://localhost:11434" # ollama server address
ollama_model: "llava"                # Key: Change this to your downloaded multimodal
model, e.g., "llava"
#.....

```

Note: Please ensure that the model specified in the configuration parameters (e.g., `llava`) can handle multimodal input.

3.2 Start and Test the Function (Text Input Mode)

Note: Using a model with a small number of parameters or running with limited memory will result in poor performance. For a better experience, please refer to the corresponding chapter in <Online Large Model (Text Interaction)>.

1. **Start the `largemodel` main program (text mode):**

Open a terminal and run the following command:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=true
```

2. **Send a text command:**

Open another terminal and run the following command:

```
ros2 run text_chat text_chat
```

Then start typing: "Based on the current environment, and save the generated environment description as a txt document".

3. **Observe the results:**

In the first terminal running the main program, you will see log output showing that the system has received the text command, called the `aiagent` tool, and then provided a prompt to the LLM. The LLM will analyze the detailed steps for calling the tool. For example, for this question, it will call the `seewhat` tool to get the screen, then provide the screen to the LLM for analysis. The parsed text will be saved in the `~/yahboom_ws/src/largemodel/resources_file/documents` folder.

4. Common Problems and Solutions

4.1 Abnormal Agent Behavior

Problem 1: The agent is stuck in an infinite loop or repeatedly executes the same tool.

Solution:

1. **Enhance the Prompt:** Add stronger constraints to the Prompt, for example: "Do not repeat the same operation," "If the observation results are the same twice in a row, please try a different tool or end the task."
2. **Switch to a more powerful LLM:** The agent's logical ability largely depends on the "IQ" of the backend LLM. A more powerful model can better understand the task and make plans.